

Topic: Javascript Basics - 3
Mentor: Sushant Goyal
Date: 03-Aug-25

Topics:

1. Hoisting
2. Arrays and its methods
3. Objects and JSON iteration

Recap:

1. Control structures
 - a. decision making
 - i. if-else statements
 - ii. is-elseif-else statement

```
if(condition) {  
}  
else {  
}
```

```
if(condition) {  
}  
else if(condition2) {  
}  
else {
```

}

2. Loops - iterating through a number of items

- a. for loop
- b. while loop
- c. do-while loop

```
for(iterator; condition; iteration update) {  
    // list of statements to execute  
}
```

```
while(condition) {  
    // list of statement to execute  
}
```

```
do {  
    // list of statements  
}  
while(condition)
```

3. Branching or Jumps

- a. switch case

```
switch(condition) {  
    case 'case1':  
        //list of statements  
        break;  
    case 'case2':  
        //list of statements
```

```

    break;
case 'case3':
    //list of statements
    break;
default:
    // list of statements to be executed if none of the cases satisfy the condition given.
}

```

4. Functions:

- a. Reusable piece of code which can be called anywhere.

```

function <function_name>() {
    // list of statements
}

```

typeof - checks if the value belongs to a particular data type and share that datatype
instanceof - it checks for the instance of the value

For a integer value type is 'number'

```

typeof 2 -> 'number'
typeof 'hello' -> 'string'
typeof true -> 'boolean'
typeof undefined -> undefined
typeof null -> 'object' // not null as expected
typeof 1n -> 'bigint'
typeof Symbol() -> 'symbol'

```

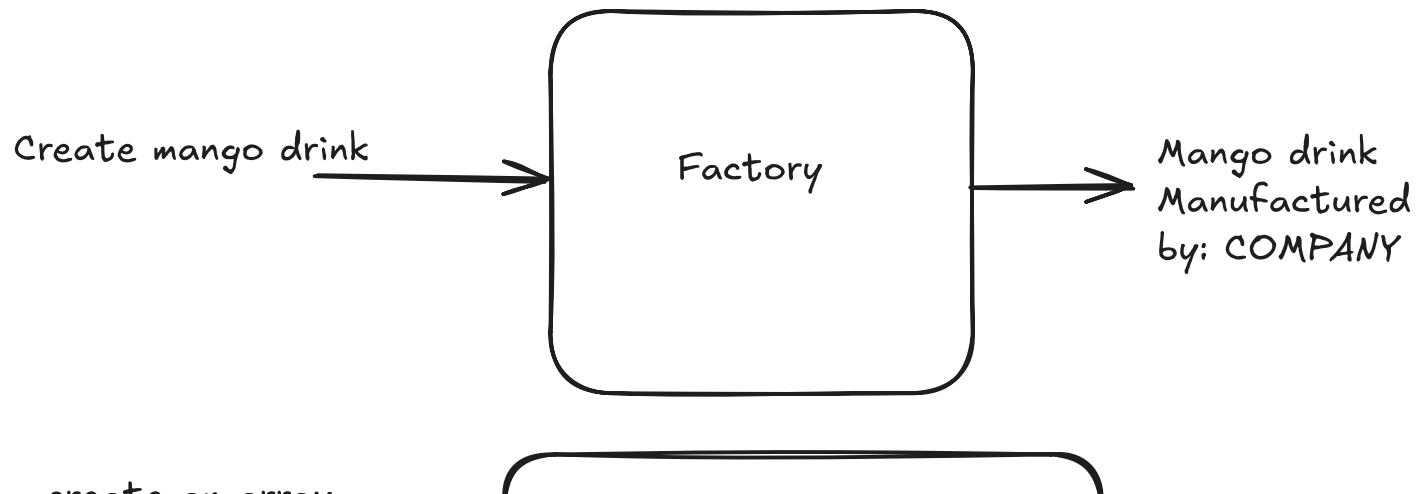
```
typeof {} -> 'object'  
typeof [] -> 'object'  
typeof function(){} -> 'function'
```

```
{  
  key1: 'value',  
  key2: 'value2'  
}
```

```
[  
  0: 'value'  
  1: 'value'  
]
```

instanceof

1. [] is an instance of Array

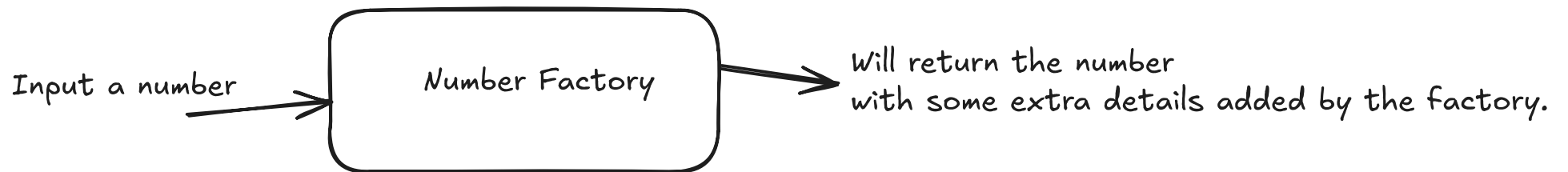




```
[] instanceof Array // true  
[] instanceof Object // true
```

Prototype -> it is a dummy version before launching it to a production version

```
{ } instanceof Object // true
```



```
let a = function() { }  
a instanceof Function // true  
a instanceof Object // true
```

```
2 instanceof Number // false  
new Number(2) instanceof Number // true
```

Hoisting:

it is a default behaviour where Js moves declaration to top of the scope

var, let and const

var - can be redeclared and reinitialized

let can be reinitiaized but in different scope

const can not be updated

Type	Hoisted?	Initialized at hoisting?	Usable before declaration?
var	Yes	Yes (undefined)	Yes (but value is undefined)
let / const	Yes	 No	 No → ReferenceError
function	Yes	Yes (entire function)	 Yes
function expression	No (only var part)	No	 No → TypeError or undefined

Scoping:

1. Global Scope -> it is present throughout the program

2. Function Scope -> Limited to that very function definition - function() {*//this is the scope*}

3. Block Scope -> if-else statements `if(condition) {//this is the scope} else {//this is the scope}`
`for(let i=0;i<5;i++){ //the scope of i will be within this loop}`

Arrays:

1. These are special kind of objects.
2. These are used to store values in a single variable.

```
let fruits=['apple', 'banana', 'mango']
```

How to access array elements:

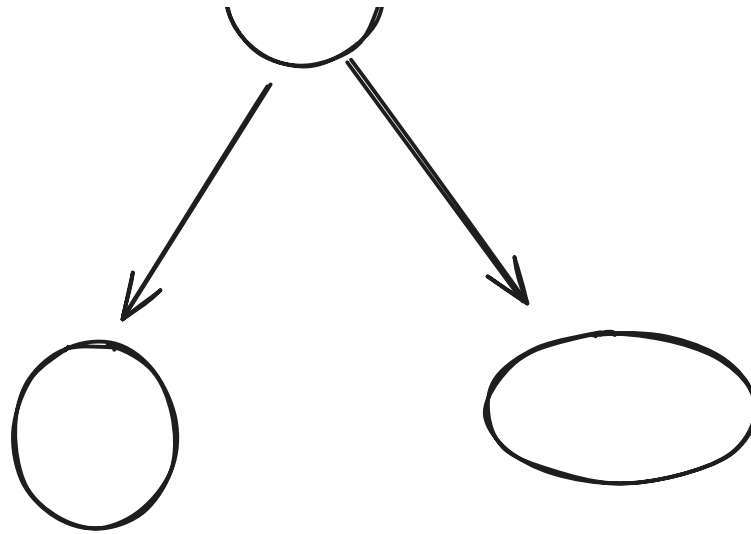
1. Using for loop with fixed iteration length
2. Using `array.length` property to find out the length and then iterating through it
3. Using `forEach` method of the array.
4. Using `map` method of the array.

Objects:

1. It is non-primitive data type that store multiple values in form of key-value pairs.

```
const car = {  
  brand: 'Toyota',  
  model: 'Camry',  
  year: 2024  
}
```





```
function Node {  
  data;  
  value;  
  next;  
}
```

```
const node1 = new Node()  
const node2 = new Node()
```

we have a product
new Product() -> brand, name, usebydate, createddate

Defining schemas within the databases and assigning values to their objects.

JSON - Javascript Object Notation

1. lightweight structure with values
2. independent of the language
3. it utilizes the same key value pair as an object

Common Use Cases:

1. Data Exchange; it is used to exchange data between server and client.
2. Storage; we can store JSON in files or databases.
3. Cross Platform; If API is created in the Python we can ask for a JSON from API to render data on React application.

Browser JS:

1. It has limited access over the system resources
2. It has limited environment control
3. Code Exposure - Public

NodeJS

1. It can access your system resources
2. We have to take care of the resources that we are managing
we are responsible for any leaks.
3. Code Exposure - Private